Determining Planar Location Via Complement-Free de Bruijn Sequences Using Discrete Optical Sensors

Edward R. Scheinerman

***Abstract*—We present a technique for determining the absolute x, y-location of a robot in a workspace. The floor of the workspace is tiled with black and white squares in a pattern based on a variant of a de Bruijn sequence. The black/white pattern on the floor is read by a modest number of inexpensive, discrete optical sensors, and a simple algorithm determines the position of the robot.**

***Index Terms*—De Bruijn sequence, Eulerian graphs, robot location.**

I. Introduction

THE BEST motion planning techniques are of little use if a robot does not know its location. In this paper, we describe a simple technique whereby a robot can determine its location with a high degree of accuracy. The central idea is to paint the floor of the workspace with a black–white pattern based on a generalization of de Bruijn sequences. Inexpensive, discrete optical sensors in the base of the robot detect the color of points on the floor, and from there the location of the robot is readily and accurately determined. The system we present here is similar to that of [1] in which discrete sensors and black/white painting are used to determine the orientation of a freely rotating ball; however, the analysis and position decoding methods are quite different.

Previous work on robot localization generally has used significantly more elaborate devices such as sonar, laser rangefinders, video cameras, odometry, etc. Some references in this area include [2]–[15].

II. De Bruijn Sequences

Let n be a positive integer. A *de Bruijn sequence* is a list of 2^n zeros and ones $(b_0, b_1, \ldots, b_{2^n-1})$ with the following property: no two length-n subsequences are the same (including the subsequences that "wrap" past the end). That is, the 2^n subsequences of the form $(b_i, b_{i+1}, \ldots, b_{i+n-1})$ (with subscript arithmetic modulo 2^n) range over all 2^n possible length-n bit vectors.

For example, the following is a de Bruijn sequence:

$$0\,0\,0\,0\,1\,1\,1\,1\,0\,1\,1\,0\,0\,1\,0\,1.$$

Note that the bit vector 1101 begins at the seventh element of the sequence and the bit vector 0100 can be found starting at the next-to-last entry.

There are well-known methods for constructing de Bruijn sequences based on the *shift graph*. Let n be a positive integer. Form a directed graph G_n with 2^n vertices labeled by all possible length-n bit vectors. We place a directed edge from a to b if the last $n-1$ bits of a are exactly the first $n-1$ bits of b. For example, from vertex 0100 there are exactly two directed edges leading to vertices 1000 and 1001. From vertex 1111 there are also exactly two edges: one to vertex 1110 and a loop back to 1111. Indeed, every vertex in G_n has out-degree and in-degree exactly 2. Thus G_n has 2^n vertices and 2^{n+1} edges. See Fig. 1.

A Hamiltonian cycle in G_n corresponds exactly to a de Bruijn sequence of length 2^n. However, it is not clear that G_n has a Hamiltonian cycle, or how to find such a cycle if it exists. However, it is easy to check that G_n is connected and, since in-degree equals out-degree at every vertex, has a Eulerian tour, i.e., a walk (beginning at any vertex) that traverses every edge exactly once (and returns to start).

We label the edges of G_n as follows. The edge from vertex $(b_1, b_2, \ldots, b_n)$ to vertex $(b_2, b_3, \ldots, b_{n+1})$ receives the label $(b_1, b_2, \ldots, b_n, b_{n+1})$. Thus, there are 2^{n+1} possible edges and every edge receives a unique $(n+1)$-bit label.

We claim that a Eulerian tour in G_n yields a de Bruijn sequence of length 2^{n+1}. Consider a pair of consecutive edges on a tour, say

$$(b_1, b_2, \ldots, b_n) \longrightarrow (b_2, b_3, \ldots, b_{n+1}) \longrightarrow (b_3, b_4, \ldots, b_{n+2}).$$

Observe that the labels on these edges are exactly $(b_1, b_2, \ldots, b_{n+1})$ and $(b_2, b_3, \ldots, b_{n+2})$, respectively. Thus, a pair of consecutive edges on a Euler tour corresponds exactly to a one-bit shift, and these shifts take us through all 2^{n+1} length-$(n+1)$ bit vectors.

For an overview of de Bruijn sequences, see [16] or a graph theory text such as [17]. See also the web site [18] where C and Pascal code is available for generating de Bruijn sequences.

III. Complement-Free de Bruijn Sequences

The tilings we develop are based on a modification of a de Bruijn sequence. We seek a sequence of 2^n zeros and ones that includes every length-$(n+1)$ bit vector *or its complement* exactly once. Let us be more precise.

For $x \in \{0, 1\}$, let $\overline{x} = 1 - x$, i.e., the complement (or negation) of x. If $b = (b_1, b_2, \ldots, b_t)$ is a sequence (vector) of 0s and 1s, then $\overline{b} = (\overline{b}_1, \overline{b}_2, \ldots, \overline{b}_t)$. We call $\overline{b}$ the *complement* of b.

Manuscript received June 2, 2000; revised July 26, 2001. This paper was recommended for publication by Associate Editor L. Kavraki and Editor S. Hutchinson upon evaluation of the reviewers' comments. This work was supported by the National Science Foundation through an Interdisciplinary Grant in the Mathematical Sciences DMS-9971696.

The author is with the Department of Mathematical Sciences, The Johns Hopkins University, Baltimore, MD 21218-2682 USA (e-mail: ers@jhu.edu).

Publisher Item Identifier S 1042-296X(01)10906-7.

1042–296X/01$10.00 © 2001 IEEE

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

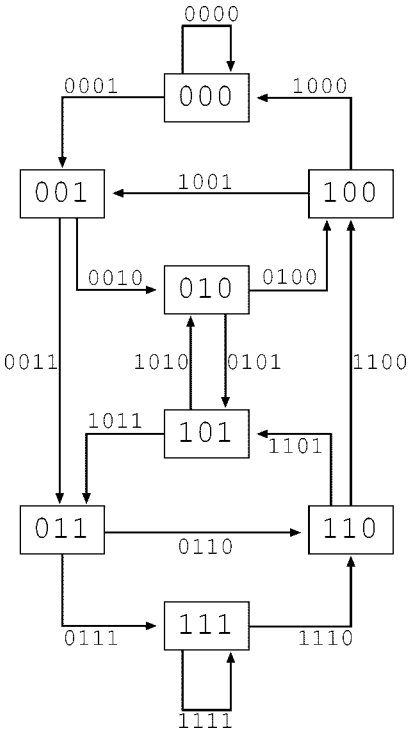


Fig. 1. The graph G_3 with edges labeled. Notice that the 16 edges in this graph receive all possible length-4 bit vectors. A Eulerian tour in this graph produces a de Bruijn sequence of length 2^4.

TABLE I

Vector	In seq.?	Complement	In seq.?
000	no	111	yes
001	no	110	yes
010	no	101	yes
011	yes	100	no

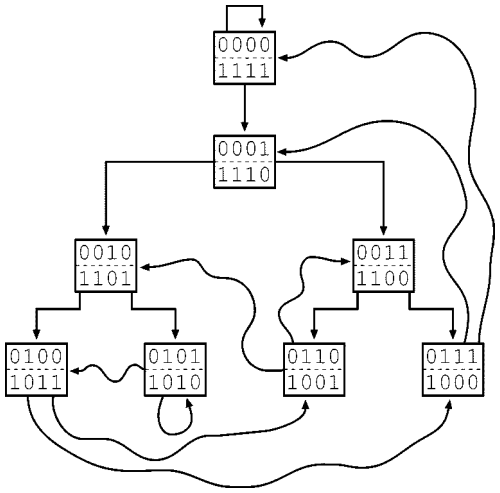


Fig. 2. The graph $\tilde{G}_4$.

We call a sequence b of 2^n zeros and ones a *complement-free de Bruijn sequence* provided for all length-$(n+1)$ bit vectors a, exactly one of a or $\overline{a}$ occurs as a subsequence of b.

Theorem 1: For every integer $n \geq 2$, there exists a complement-free de Bruijn sequence of length 2^n.

For example, for $n = 2$ consider the sequence 0111. There are $2^3 = 8$ length-3 bit vectors. Table I lists all of them, paired with their complements, and indicates which of the 8 are contained in the sequence.

Proof: Let $n \geq 2$ be given and consider the shift graph G_n. We form a new graph $\tilde{G}_n$ by identifying every vertex a with its complement $\overline{a}$. That is, the two vertices a and $\overline{a}$ are merged into a single vertex. Note that if there is a directed edge $a \rightarrow b$ in G_n, then there must also be a directed edge $\overline{a} \rightarrow \overline{b}$. These two edges collapse into a single edge in $\tilde{G}_n$. Furthermore, the labels on the two edges $a \rightarrow b$ and $\overline{a} \rightarrow \overline{b}$ must be complements of each other since the label on $a \rightarrow b$ is formed from the bits of a and b in the same manner as the label on $\overline{a} \rightarrow \overline{b}$ is formed from the bits of $\overline{a}$ and $\overline{b}$.

We label the merged vertex with both labels a and $\overline{a}$. Note that one of these two labels first bit must be a 0. We imagine that the merged vertex's "top" label is the one that begins with a 0 and the "bottom" label is the one that begins with a 1. The graph $\tilde{G}_4$ appears in Fig. 2. We use the notation $(a;\overline{a})$ to represent the pair of labels on a merged vertex; because a is first, it is the top vertex and its leading bit must be a 0.

Notice that there are 2^{n-1} vertices in $\tilde{G}_n$. Furthermore, every vertex has in-degree and out-degree equal to 2, so there are 2^n edges in $\tilde{G}_n$. Finally, since G_n is connected, so is $\tilde{G}_n$. Thus, $\tilde{G}_n$ has a Eulerian tour.

We distinguish two types of edges in the graph $\tilde{G}_n$: *straight* and *twisted.* Two edges $a \rightarrow b$ and $\overline{a} \rightarrow \overline{b}$ in G_n collapse into a single edge in $\tilde{G}_n$. If the $a \rightarrow b$ edge points from the top of $(a;\overline{a})$ to the top of $(b;\overline{b})$ or if it points from bottom to bottom, we call the merged edge *straight.* Otherwise (the edges runs top-to-bottom and bottom-to-top) we call the merged edge *twisted.* This is illustrated in Fig. 3.

We can determine whether a merged edge is twisted or straight by examining the second bit of the labels. Suppose the top label on a merged vertex is $(0, a_2, \ldots, a_n)$ and the bottom label is $(1, \overline{a}_2, \ldots, \overline{a}_n)$. If $a_2 = 0$, then the two edges emanating from this vertex are straight [they lead to vertices whose top labels are $(a_2, \ldots, a_n, 0)$ and $(a_2, \ldots, a_n, 1)$]. However, if $a_2 = 1$, then the two edges leading out of this merged vertex are both twisted [leading to vertices whose top labels are $(\overline{a}_2, \ldots, \overline{a}_n, 0)$ and $(\overline{a}_2, \ldots, \overline{a}_n, 1)$]. Since exactly half the top labels have second bit equal to 0, we conclude that half of the edges in $\tilde{G}_n$ are straight and half are twisted.

In Fig. 2, straight edges are drawn using straight (polygonal) segments, while twisted edges are drawn as smooth curves. We denote a straight edge from $(a;\overline{a})$ to $(b;\overline{b})$ by a straight arrow $\rightarrow$ and twisted edges by $\rightsquigarrow$.

Also, we label the edges of $\tilde{G}_n$ by a pair of labels $(b;\overline{b})$. These are the labels inherited from the two edges of G_n that were merged together.

Consider a Eulerian tour through G_n. As we traverse each edge of $\tilde{G}_n$, we add one bit to a sequence that becomes a complement-free de Bruijn sequence of length 2^n.

The tour begins at vertex $(0 \cdots 0; 1 \cdots 1)$ and the growing sequence begins with n zeros.

As we move from vertex to vertex, we consider ourselves to be in one of two states: *up* or *down.* When we are in the "up"

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

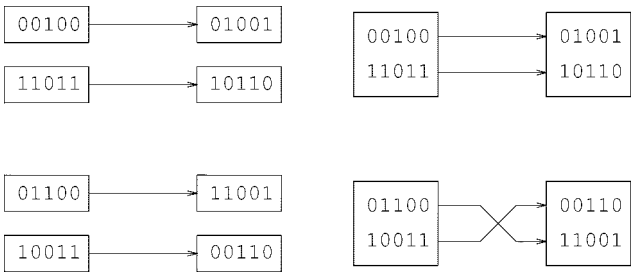


Fig. 3. Merging vertices in G_5 resulting in a straight edge (top) or a twisted edge (bottom) in $\tilde{G}_5$.

state, we imagine that our attention is focused on the top label, and conversely for the "down" state.

There are four cases to consider.

- We traverse an edge of the form $(00a; 11\overline{a}) \to (0a0; 1\overline{a}1)$:
  If we are in the top state, we add a 0 to the growing sequence and we stay in the top state.
  If we are in the bottom state, we add a 1 to the growing sequence and we stay in the bottom state.
- We traverse an edge of the form $(00a; 11\overline{a}) \to (0a1; 1\overline{a}0)$:
  If we are in the top state, we add a 1 to the growing sequence and we stay in the top state.
  If we are in the bottom state, we add a 0 to the growing sequence and we stay in the bottom state.
- We traverse an edge of the form $(01a; 10\overline{a}) \rightsquigarrow (0\overline{a}1; 1a0)$:
  If we are in the top state, we add a 0 to the growing sequence and we switch to the bottom state.
  If we are in the bottom state, we add a 1 to the growing sequence and we switch to the top state.
- We traverse an edge of the form $(01a; 10\overline{a}) \rightsquigarrow (0\overline{a}0; 1a1)$:
  If we are in the top state, we add a 1 to the growing sequence and we switch to the bottom state.
  If we are in the bottom state, we add a 0 to the growing sequence and we switch to the top state.

Note that straight edges preserve the state, but twisted edges toggle the state. Also, the bit we add to the growing sequence corresponds to the new bit added to the vertex label on either the top or the bottom depending on the resulting state.

Because there are an even number of twisted edges, when we complete the Eulerian tour, the state at the end of the tour matches the state at the beginning of the tour.

For example, let $n = 4$ (see Fig. 2). Table II describes the state of this algorithm at each step. Because there are an even number of twisted edges (because $n > 2$) the algorithm begins and ends in the "up" state. We take the first 2^n terms of the complete sequence. In our example, this is 0000 0100 0110.

For the $n = 4$ example, the 5-bit strings we meet and their complements are given in Table III. Careful examination of the table confirms that all 2^5 are present exactly once and so the sequence contains 2^4 length-5 bit vectors, and a vector is present iff its complement is absent.

Finally, to verify that this procedure gives a complement-free de Bruijn sequence, consider the Euler tour in $\tilde{G}_n$ and lift it back to G_n. That is, the tour begins at the top of $(0\cdots0; 1\cdots1)$; we bring this back to the vertex $0\cdots0$ in G_n. In general, if the tour in $\tilde{G}_n$ is in the state "up" for vertex $(a, \overline{a})$ we lift back to a in G_n, while if the tour is in the "bottom" state we lift to $\overline{a}$. This gives a closed trail in G_n that uses exactly one from each pair of complementary edges $a \to b$ and $\overline{a} \to \overline{b}$. The lifted tour traverses exactly half the edges in G_n (one for each complementary pair) and returns to start (because there are an even number of twisted edges in $\tilde{G}_n$). Thus, the partial tour of G_n yields a length-2^n complement-free de Bruijn sequence. ■

TABLE II
GROWING THE SEQUENCE

Vertex	State	Growing Sequence
0000;1111	up	0000
0001;1110	up	00001
0010;1101	up	000010
0100;1011	up	0000100
0110;1001	down	00001001
0010;1101	up	000010010
0101;1010	up	0000100101
0101;1010	down	00001001010
0100;1011	up	000010010100
0111;1000	down	0000100101000
0001;1110	up	00001001010001
0011;1100	up	000010010100011
0110;1001	up	0000100101000110
0011;1100	down	00001001010001100
0111;1000	down	000010010100011000
0000;1111	up	0000100101000110000
0000;1111	up	00001001010001100000

TABLE III
CHECKING THE SEQUENCE

Subseq	Comp	Subseq	Comp
00001	11110	01000	10111
00010	11101	10001	01110
00100	11011	00011	11100
01001	10110	00110	11001
10010	01101	01100	10011
00101	11010	11000	00111
01010	10101	10000	01111
10100	01011	00000	11111

IV. PAINTING THE FLOOR AND DECODING POSITION

We now apply complement-free de Bruijn sequences to solve the robot location problem. Let $n \geq 2$ be an integer. Let b be a complement-free de Bruijn sequence of length 2^n.

Imagine that the workspace is a square consisting of $2^n \times 2^n$ tiles. We color tile i, j black if $b_i + b_j$ is even; otherwise, we color it white. In other words, the i, j-tile's color is given by $(b_i + b_j) \bmod 2$. Such a painting is depicted in Fig. 4.

In the base of the robot there are $2n + 1$ sensors; $n + 1$ of the sensors are aligned with the rows of the tiling and $n + 1$ of the sensors are aligned with the columns. (The total is $2n + 1$ because a row and a column have a common sensor.) For example, the sensors may form an L shape along two edges of the robot. Each sensors determines the color of the point immediately below and returns a 0 for black and a 1 for white.

To decode the robot's position, consider first the sensors aligned with a row of the floor. The bit pattern we read is either $(a_0, a_1, \ldots, a_n)$ or $(\overline{a}_0, \overline{a}_1, \ldots, \overline{a}_n)$. Exactly one of these two patterns is present in b, and therefore we can determine horizontal position. Likewise, we can determine vertical position by examining the bit pattern from the column sensors.

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

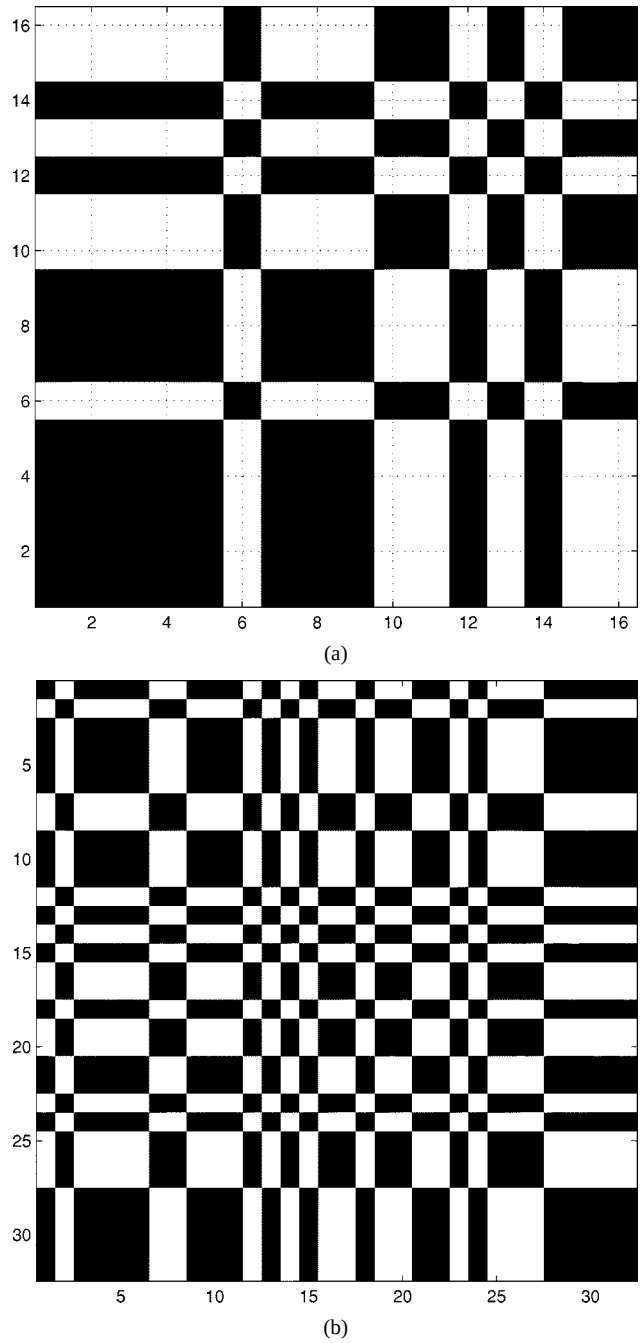


Fig. 4. (a) Floor painting based on the complement-free de Bruijn sequence 0000 1001 0100 0110 and (b) another based on a 2^5-long sequence.

V. Implementation Details

A. An Example

Imagine a 100×100 m^2 work space (e.g., a large factory floor) over which robots move. We require that the location of the robot be known to 1-cm accuracy. This implies that we paint the floor with a $10\,000 \times 10\,000$ grid (or finer). Since $2^{14} = 16\,384 > 10\,000$, we create a 2^{14}-long complement-free de Bruijn sequence d. We fit the bottom of the robot with 29 sensors as described above (15 in a row and 15 in a column).

To decode the robot's position, we read the row and column sensors, and then we determine where those bit patterns (or their complements) appear in d. The simplest method is to store the sequence d in `ram` (or `rom`) on the robot. This requires $2^{14}/8 = 2048$ bytes, or just 2K of storage. The robot can scan through the entire list quickly. Alternatively, as the robot moves, the position can be determined by considering a few subsequences near the last-known position. A faster, but moderately more expensive solution, is to store a look-up table that gives, for all 2^{15} possible inputs, the location in d where the bit pattern (or complement) occurs. Such a table would have 2^{15} entries, each of 2 bytes. Such a table would require only 64K bytes.

The (one-time) computational effort to create d is reasonable. There are efficient methods (e.g., Fleury's algorithm) for finding Eulerian trails in graphs. We wrote a computer program to find complement-free de Bruijn sequences (the code is in `C++` and uses the `LEDA` package). The running time to produce a 2^{14}-long complement-free de Bruijn sequence on a 450-MHz Linux computer was only 36 s.

B. Local Decoding

The complement-free de Bruijn tiling can be used as a local xy-position decoder as well. Because the de Bruijn sequence is, in fact, a circular sequence, we can cover a large works space by repeating (say) a 16×16 pattern (see Fig. 4) on a tile, and then use that tile repeatedly to cover a workspace. We would manually give the robot its initial position. From there, it could track its progress using only 9 sensors, and keep track (by counting) on which 16×16 tile it is currently positioned.

C. Speed of Sensors

Discrete optical sensors can react quickly. For example, Optoelectronics's optical sensor model QRD1114 slowest reaction time is 50×10^{-6} seconds. Consider a robot, using these sensors, moving over a 1 cm tiling, and—to be confident in our position reading—we want the sensors to give a constant reading that is ten times the sensor's reaction time, i.e., 500×10^{-6} s. This gives a maximum robot speed of 1 cm/(500×10^{-6} s) = 20 m/s, which is 72 kph or about 45 mph.

D. Alignment

In order for the sensor readings to be properly decoded, it is important that the robot remains oriented with respect to the grid of the floor painting. It is possible to use the same sensors to keep the robot aligned to the grid. As the robot moves (say) vertically, some of the sensors will detect a change in the black/white color they read. These changes should all occur simultaneously. (There will be a brief period of ambiguity as the sensor moves from a tile of one color to another, but because the sensors react quickly, the transition time will be negligible.)

If the sensor changes are not simultaneous, the robot can then rotate slightly in the appropriate direction, and move again. In this way, the robot will be able to align itself (or its sensors) to the grid.

VI. A Random Approach

Let n, N be fixed positive integers and let $a = (a_0, a_1, \ldots, a_{N-1})$ be a sequence of 0s and 1s. Let $a(i)$ denote the n-bit subsequence $(a_i, a_{i+1}, \ldots, a_{i+n-1})$ (where

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

subscript addition is taken modulo N). The sequence is *repetition-free* if $a(i) \neq a(j)$ for all $i \neq j$ and it is *complement-free* if $a(i) \neq \overline{a(j)}$ for all $i \neq j$. Note that it is impossible to have $a(i) = \overline{a(i)}$. We assume that $N > 2n$; this prevents two length-n subsequences from intersecting at both ends.

If a is a complement-free de Bruijn sequence, then a is both repetition-free and complement-free, and $N = 2^{n-1}$ is the greatest length possible for any sequence with both properties. Suppose, instead, the sequence a is generated at random. How large may we take N and have, with high probability, a sequence that is repetition- and complement-free? Conversely, if we are given N (the sequence length) how small may we take n (the word size) and still be confident that the sequence is repetition-free and complement-free.

Given a, and $i \neq j$, define the random variables X_{ij} and Y_{ij} as follows:

$$X_{ij} = \begin{cases} 1, & \text{if } a(i) = a(j) \\ 0, & \text{otherwise} \end{cases}$$

and

$$Y_{ij} = \begin{cases} 1, & \text{if } a(i) = \overline{a(j)} \\ 0, & \text{otherwise.} \end{cases}$$

Let Z be the sum of the X_{ij} and Y_{ij} over all $i < j$. Note that a is repetition- and complement-free if and only if $Z = 0$. By Markov's inequality,

$$\Pr(Z > 0) \leq E(Z) = \sum_{i<j} [E(X_{ij}) + E(Y_{ij})].$$

To evaluate $E(Z)$, we calculate $E(X_{ij})$ and $E(Y_{ij})$.

Lemma 2: With the above notation, and for all $i \neq j$, $E(X_{ij}) = E(Y_{ij}) = 2^{-n}$.

Proof: The result is obvious in case the subsequences $a(i)$ and $a(j)$ do not overlap. Without loss of generality, we may take $i = 0$ and $j = t$ where $0 < t < n$. The two subsequences span a total of $n + t$ bits, so there are 2^{n+t} choices for these bits. In how many of these cases do we have $a(0) = a(t)$? We may take the first t bits of $a(0)$ arbitrarily. After that, each successive bit must be exactly the same as the bit t places earlier. Thus, there are precisely 2^t possibilities in which $a(0) = a(t)$, and so $X_{0t} = 2^t/2^{n+1} = 2^{-n}$.

The analysis for Y_{0t} is analogous. There are 2^t choices for the first t bits. After that, every bit must be the complement of the bit t places earlier and we have $Y_{0t} = 2^{-n}$. ∎

Using Lemma 2, we have

$$\Pr(Z > 0) \leq E(Z) < N^2 2^{-n}$$

and so, if we take $n > 2\lg N$, we have $\Pr(Z > 0) < 1$ and so $\Pr(Z = 0) > 0$.

Therefore, we may use a randomly generated sequence of length N to be the basis for a floor painting for robot location. Such a system would use (roughly) $4\lg N$ sensors ($2\lg N$ is each direction) compared to (about) $2\lg N$ for a painting based on complement-free de Bruijn sequences. This factor of 2 savings is modest because the discrete optical sensors are inexpensive. Indeed, the random painting has an advantage that we now explore.

A. Orientation Woes

The location method based on complement-free de Bruijn sequences has the following property. Because there are $2n + 1$ sensors and there are $2^n \times 2^n = 2^{2n}$ possible locations for the robot, exactly half of the possible sensor inputs are valid and half are invalid. If the robot is lifted from the plane, rotated 90° or 180°, and replaced in the plane, the robot mistakenly may decode a valid set of inputs. Consequently, it will not know that it is out of alignment to the grid and will incorrectly compute its location. One solution to this difficulty is to fit the robot with a compass. Such a compass does not need to be highly accurate; the ability to determine magnetic north within (say) 20° would be sufficient. However, in an industrial setting, stray magnetic fields may play havoc with the compass. We present an alternative solution that only uses the optical sensors.

As before, let $a = (a_0, a_1, \ldots, a_{N-1})$ be a sequence of 0s and 1s, and let $a(i)$ denote the n-bit subsequence $(a_i, a_{i+1}, \ldots, a_{i+n-1})$ (where subscript addition is taken modulo N).

Given a sequence $b = (b_1, b_2, \ldots, b_s)$, define the *reversal* of b to be the sequence $\rho(b) = (b_s, b_{s-1}, \ldots, b_2, b_1)$. We call the sequence a *reversal-free* if $a(i) \neq \rho[a(j)]$ for all i, j (including $i = j$). We call a *complement-reversal-free* if $a(i) \neq \overline{\rho[a(j)]}$ for all i, j. Finally, we say a is *trouble-free* if a has all four properties: repetition-free, complement-free, reversal-free, and complement-reversal-free.

We show that if n is large enough (or N is small enough), then a random sequence a is, with high probability, trouble-free. We then show how to create a floor painting based on a trouble-free sequence to solve the robot location problem that is robust against 90° and 180° changes in orientation.

B. Random Trouble-Free Sequences

Let $a = (a_0, a_1, \ldots, a_{N-1})$ be a sequence of 0s and 1s. Define indicator random variables U_{ij} and W_{ij} as follows:

$$U_{ij} = \begin{cases} 1, & \text{if } a(i) = \rho[a(j)] \\ 0, & \text{otherwise} \end{cases}$$

and

$$W_{ij} = \begin{cases} 1, & \text{if } a(i) = \overline{\rho[a(j)]} \\ 0, & \text{otherwise.} \end{cases}$$

Note that, if $a(i)$ and $a(j)$ have no overlap, then $E[U_{ij}] = E[W_{ij}] = 2^{-n}$. In case they overlap, we have the following result.

Lemma 3: With the above notation and for $0 \leq t < n$, we have

$$E[U_{0t}] = 2^{\lceil n - t/2 \rceil - n}.$$

Proof: Let x denote the first t bits of $a(0)$ and let y denote the last $n - t$ bits. In order to have $a(0) = \rho[a(t)]$ we need precisely that $y = \rho(y)$; otherwise the bits in x and y are arbitrary. All t bits of x may be freely chosen, but only the first $\lceil (n-t)/2 \rceil$ bits of y are free. Thus, of the $n+t$ bits in the two subsequences, we can choose $t + \lceil (n-t)/2 \rceil$ bits freely. Therefore, the probability that $a(0) = \rho[a(t)]$ is exactly $2^{\lceil (n-t)/2 \rceil - n}$. ∎

Lemma 4: With the above notation and for $0 \leq t < n$, we have

$$E[W_{0t}] = \begin{cases} 2^{(n-t)/2 - n}, & \text{in case } n - t \text{ is even,} \\ 0, & \text{otherwise.} \end{cases}$$

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

Proof: As in the proof of Lemma 3, let x denote the first t bits of $a(0)$ and let y denote the final $n-t$ bits. In order to have $a(0) = \overline{\rho[a(t)]}$ we need precisely that $y = \overline{\rho(y)}$. This is impossible if y has an odd number of bits. Otherwise, of the $n+t$ bits in $a(0)$ and $a(t)$, there are t free bits in x and $(n-t)/2$ free bits in y and the result follows. ∎

Theorem 5: Let $N > 2n$ and let a be a random, length-N sequence of 0s and 1s. If $n \geq 2\lg N + 7$, then a is trouble-free (for subsequences of length n) with probability greater than $1/10$.

Thus, if $n \geq 2\lg N + 7$ (and $n < N/2$), at least 10% of all length-N sequences of zeros and ones are trouble-free (for subsequences of length n). Therefore, such a sequence must exist.

Proof: Let U_{ij}, W_{ij}, X_{ij}, and Y_{ij} be defined as above. Let

$$U = \sum_{i \leq j} U_{ij}, \quad W = \sum_{i \leq j} W_{ij}$$
$$X = \sum_{i < j} X_{ij}, \quad Y = \sum_{i < j} Y_{ij}$$

and let $Z = U + W + X + Y$. Note that a is trouble-free if and only if $Z = 0$. From Lemma 2, we have

$$E(X) = E(Y) = \binom{N}{2} 2^{-n}.$$

From Lemmas 3 and 4, we have

$$\begin{aligned}\sum_{t=0}^{n-1} E(W_{0t}) \leq \sum_{t=0}^{n-1} E(U_{0t}) &\leq \sum_{t=0}^{n-1} 2^{(1-t-n)/2} \\ &\leq 2^{(1-n)/2} \sum_{t=0}^{n-1} 2^{-t/2} < \frac{2^{(1-n)/2}}{1-2^{-1/2}} \\ &< 5 \cdot 2^{-n/2}.\end{aligned}$$

Therefore

$$\begin{aligned}E(W) \leq E(U) &\leq \sum_{i \leq j} E(U_{ij}) \\ &\leq N \sum_{t=0}^{n-1} E(U_{0t}) + \binom{N}{2} 2^{-n} \\ &\leq 5N2^{-n/2} + \binom{N}{2} 2^{-n}.\end{aligned}$$

And so

$$\begin{aligned}E(Z) &= E(X) + E(Y) + E(U) + E(W) \\ &\leq 4\binom{N}{2} 2^{-n} + 10N2^{-n/2} \\ &\leq 2N^2 2^{-n} + 10N2^{-n/2} \\ &= 2(N2^{-n/2})^2 + 10(N2^{-n/2}).\end{aligned}$$

If $n \geq 2\lg N + 7$, then $N2^{-n/2} \leq 1/\sqrt{128}$, and so

$$\begin{aligned}\Pr(Z > 0) &\leq E(Z) \\ &\leq 2\left(\frac{1}{\sqrt{128}}\right)^2 + 10\left(\frac{1}{\sqrt{128}}\right) \\ &= \frac{1 + 40\sqrt{2}}{64} < \frac{9}{10}.\end{aligned}$$

Therefore, $\Pr(Z = 0) > 1/10$. ∎

C. Orientation Solution

We now show how to use a trouble-free sequence to produce a floor painting on which a robot can determine its location, even if it is rotated 90° or 180° from its "home" orientation.

Let a be a trouble-free (for word size n) sequence of length $2N$. By Theorem 5, such a sequence must exist for $n \geq 2\lg N + 8$. Partition the floor of the workspace into an $N \times N$ grid. Color the square at location i, j white if $a_i + a_{N+j}$ is even and black if $a_i + a_{N+j}$ is black. The robot is fitted with $2n-1$ sensors (n in each direction with one overlap). When a row of sensors reads a bit pattern from the floor, the bit pattern must be one of $a(k)$, $\overline{a(k)}$, $\rho(a(k))$, or $\overline{\rho(a(k))}$ for exactly one value of k. The subsequence $a(k)$ either appears in the first half or the second half of the sequence a. From this, the robot determines if the sensors are horizontally or vertically aligned, and computes its horizontal (or vertical) position accordingly. It calculates its position along the perpendicular axis in exactly the same manner.

Let us revisit the example we considered earlier: a 100×100 meter workspace in which we want to locate robots to 1-cm accuracy. We generate random 0, 1 sequences of length $N = 20\,000$ until we achieve a sequence that is trouble-free for subsequences of size $\lceil 2\lg N + 7 \rceil = 36$, or less. A 20 000-long sequence of zeros and ones can be stored using less than 3K bytes of memory.

The bound of 36 is conservative; in practice we can generate a 20 000-long sequence that is trouble-free for subsequences of size 30.

D. Finding a Trouble-Free Sequence

Suppose we wish to find a sequence of length N that is trouble-free for length-n subsequences. By Theorem 5, if $n \geq 2\lg N + 7$, then a random sequence of N bits will have the required property with probability greater than 10%. It is simple to generate the random sequence, and then to explicitly check if it is trouble-free. To check if a sequence is trouble-free, we build a large list by including, for each subsequence w, the four sequences w, $\rho(w)$, $\overline{w}$ and $\overline{\rho(w)}$. We then sort the list and look for repetitions.

For $N = 20\,000$ and $n = 36$, this process takes only a few seconds on a 450-MHz computer.

If the sequence we generate is not trouble-free, we simply generate a new sequence and try again. The probability we have not found a trouble-free sequence after k attempts is less than 0.9^k. For $k = 50$, this is less than 1%. Thus, in a few minutes, we can generate the desired sequence. This is a one-time calculation; once the sequence is found, it need not be recomputed.

E. Local Encoding with Trouble-Free Sequences

Trouble-free sequences can also be used for a local encoding scheme. We create a modest-length trouble-free sequence (say, $N = 64$) and use $N \times N$ tiles as described above in Section V (see the *Local decoding* subsection).

VII. Costs

It may be argued that this location system replaces reasonably expensive sensors (such as laser range-finders) with an unreasonably expensive floor. In short, we have transferred the cost

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.

from the robot to the workspace and come out in debt. This is a fair assessment if the workspace is populated by only a single robot. If, however, there are *many* robots sharing this workspace, then the cost is amortized. The marginal cost of adding a robot to this workspace is greatly reduced.

If the workspace is to be populated by only a single robot (or just a few), then the relative (as opposed to absolute) location encoding system is economical. A short (say 32×32) version of the pattern could be printed (mass produced) on a tile, and the workspace be covered with these tiles.

VIII. Summary

We have presented a technique for encoding planar position. The central idea is an efficient black/white painting of the plane so that x, y-position can be quickly determined using simple, discrete optical sensors. The painting is based on variants of de Bruijn sequences. We propose this encoding method as a technique for robots to determine their location in a workspace, but it can be used more broadly for any device that moves through a plane.

Acknowledgment

The author thanks G. Chirikjian, D. Stein, and D. Rus for many helpful comments and suggestions. He is also grateful to his colleagues in the Mechanical Engineering Department at Hopkins for their hospitality during the 1999–2000 academic year.

References

[1] E. R. Scheinerman, G. S. Chirikjian, and D. Stein, "Encoders for spherical motion using discrete optical sensors," in *Algorithmic and Computational Robotics: New Directions (Proc. 2000 Workshop on the Algorithmic Foundations of Robotics)*, B. R. Donald, K. M. Lynch, and D. Rus, Eds: A. K. Peters, 2001, pp. 309–315.

[2] J. Borenstein, H. R. Everett, and L. Feng, *Navigating Mobile Robots: Systems and Technique*. Natick, MA: A. K. Peters, 1996.

[3] A. J. Briggs, D. Scharstein, and S. D. Abbott, "Reliable mobile robot navigation from unreliable visual cues," in *Proc. Fourth Int. Workshop on Algorithmic Foundations of Robotics (WAFR 2000)*, 2000.

[4] J. A. Castellanos and J. D. Tardós, *Mobile Robot Localization and Map Building: A Multisensor Fusion Approach*. Norwell, MA: Kluwer, 2000.

[5] G. Dudek and M. Jenkin, *Computational Principles of Mobile Robotics*. Cambridge, U.K.: Cambridge Univ. Press, 2000.

[6] D. Fox, W. Burgard, F. Dellaert, and S. Thrun, "Monte Carlo localization: Efficient position estimation for mobile robots," in *Proc. National Conf. on Artificial Intelligence*, 1999.

[7] D. Fox, W. Burgard, H. Kruppa, and S. Thrun, "A probabilistic approach to collaborative multi-robot localization," Auton. Robots, 2000, to be published.

[8] D. Fox, W. Burgard, and S. Thrun, "Active Markov localization for mobile robots," *Robot. Auton. Syst.*, vol. 25, pp. 195–207, 1998.

[9] ——, "Markov localization for mobile robots in dynamic environments," *J. Artif. Intell. Res.*, pp. 391–342, 1999.

[10] I. Horswill, "Polly: A vision-based artificial agent," in *Proc. 11th National Conf. on Artificial Intelligence*, 1993, pp. 824–829.

[11] J. Howell and K. Kotay, "Landmarks for absolute localization," Dartmouth College, Computer Science, Hanover, NH, Tech. Report TR2000-364, Mar. 2000.

[12] J. L. Jones and A. M. Flynn, *Mobile Robots: Inspiration to Implementation*. Natick, MA: A. K. Peters, 1993.

[13] D. Lee, *The Map-Building and Exploration Strategies of a Simple Sonar-Equipped Robot: An Experimental, Quantitative Evaluation*. Cambridge, U.K.: Cambridge Univ. Press, 1996.

[14] J. J. Leonard and H. F. Durrant-Whyte, *Directed Sonar Sensing for Mobile Robot Navigation*. Norwell, MA: Kluwer, 1992.

[15] I. Ulrich and I. Nourbakhsh, "Appearance-based place recognition for topological localization," in *Proc. ICRA 2000*, vol. 2, Apr. 2000.

[16] K. H. Rosen, Ed., *Handbook of Discrete and Combinatorial Mathematics*. Boca Raton, FL: CRC Press, 2000, pp. 155–156.

[17] D. B. West, *Introduction to Graph Theory*. Englewood Cliffs, NJ: Prentice-Hall, 1996, pp. 92–93.

[18] F. Ruskey. The (combinatorial) object server. [Online]. Available: http://www.theory.csc.uvic.ca/~cos/

[19] H. R. Everett, *Sensors for Mobile Robots: Theory and Application*. Natick, MA: A. K. Peters, 1995.



Edward R. Scheinerman received the Sc.B. degree from Brown University, Providence, RI, and the Ph.D. degree in mathematics from Princeton University, Princeton, NJ, in 1984.

From there, he joined the Department of Mathematical Sciences of the G.W.C. Whiting School of Engineering of the Johns Hopkins University, Baltimore, MD, where he currently serves as chair of that department. He also holds a joint appointment in the Computer Science Department. During the 1999–2000 academic year, he was a visitor in Hopkins' Mechanical Engineering Department. His research interests are in discrete mathematics, especially graph theory and partially ordered sets.

Dr. Scheinerman is a member of the American Mathematical Society, the Mathematics Association of America, and the Society for Industrial and Applied Mathematics.

Authorized licensed use limited to: Lakehead University. Downloaded on September 25,2026 at 18:15:10 UTC from IEEE Xplore. Restrictions apply.